

NumPy 速查手册（第 1-7 节简化版）

用途：背诵 / 速查 / 面试前复习

特点：只讲是什么、干什么用，不举例不展开

配套：详细版见《NumPy完整笔记_第1-7节.md》

第 1 节：NumPy 是什么

核心定位

- **NumPy = 张量数据结构 + 高速运算引擎**
- 让你像写数学公式一样写代码
- Pandas / Scikit-learn / PyTorch / TensorFlow 的底层

数据结构

- `ndarray`: N 维数组
- 一维 = 向量
- 二维 = 矩阵
- 三维及以上 = 张量

与 Python list 的区别

- 运算符是逐元素（不是拼接）
 - 底层 C 实现，比循环快 50-100 倍
 - 支持矩阵运算
-

第 2 节：创建数组

从已有数据创建

- `np.array(list)`: 从 Python list 创建

特殊矩阵（形状参数用 tuple）

- `np.zeros(shape)`: 零矩阵
- `np.ones(shape)`: 全 1 矩阵
- `np.eye(n)`: 单位矩阵 I_n

等差数列

- `np.arange(start, stop, step)`: 左闭右开，第三个参数是步长，默认 int
- `np.linspace(start, stop, num)`: 左闭右闭，第三个参数是点数，默认 float

随机数

- `np.random.rand(shape)`: $U(0, 1)$ 均匀分布
- `np.random.randn(shape)`: $N(0, 1)$ 标准正态分布
- `np.random.uniform(low, high, size)`: 任意区间均匀分布
- `np.random.normal(loc, scale, size)`: 任意正态分布
- `np.random.randint(low, high, size)`: 随机整数
- `np.random.seed(n)`: 设种子

命名记忆

- `randn` 的 **n** = **n**ormal（正态）
-

第 3 节：核心属性

4 个核心属性（不带括号）

- `A.shape`: 形状（tuple）

- `A.ndim`: 维度数 (int)
- `A.dtype`: 数据类型
- `A.size`: 总元素个数

属性 vs 方法

- **属性** (不带括号): 描述信息
- **方法** (带括号): 执行动作

常见 dtype

- `int64`: 默认整数
- `float64`: NumPy 默认浮点
- `float32`: 深度学习默认
- `bool`: 布尔数组

dtype 转换

- `.astype(新类型)`: 转换数据类型

形状陷阱

- `(3,)`、`(1, 3)`、`(3, 1)` 是不同形状
- 单元素 tuple 必须加逗号

第 4 节：运算

逐元素运算

- `+` `-` `*` `/`: 逐元素四则运算
- `**`: 逐元素幂运算
- `>` `<` `==`: 逐元素比较

矩阵乘法

- `A @ B`: 矩阵乘法 (推荐)
- `np.matmul(A, B)`: 等价
- `np.dot(A, B)`: 等价

* VS @

- `*` = Hadamard 积 (逐元素乘)
- `@` = 矩阵乘法

转置与形状

- `A.T`: 转置
- `A.reshape(new_shape)`: 改变形状
- `-1`: 自动推断该维度

线代操作

- `np.linalg.inv(A)`: 求逆
- `np.linalg.det(A)`: 行列式
- `np.linalg.eig(A)`: 特征值/特征向量 (返回 tuple)
- `np.linalg.eigh(A)`: 对称矩阵专用
- `np.linalg.norm(x)`: 范数 (默认 L2)
- `np.linalg.solve(A, b)`: 解线性方程组 (推荐)
- `np.linalg.lstsq(A, b)`: 最小二乘解

范数选项

- `ord=1`: L1 范数
- `ord=2`: L2 范数 (默认)
- `ord=np.inf`: L^∞ 范数

特征向量取法

- `eigvals, eigvecs = np.linalg.eig(A)`: 元组拆包
- `eigvecs[:, i]` 是第 i 个特征向量 (按列存)

聚合函数

- `.sum/mean/std/var`
- `.min/max`
- `.argmin/argmax`:最值的索引

axis 参数

- **axis = 要被消掉的维度**
- `axis=0`:消掉行 → 每列汇总
- `axis=1`:消掉列 → 每行汇总

数学函数（全部逐元素）

- `np.exp/log/log2/log10`
- `np.sqrt/sin/cos/abs`
- `np.maximum(a, b)/np.minimum(a, b)`

第 5 节：索引与切片

一维索引

- 索引从 **0 开始**，负数从末尾倒数

一维切片

- 语法: `a[start:stop:step]`，左闭右开
- `a[::-1]`:反转

二维索引

- `A[i, j]`:单元素（**推荐**）
- `A[i, :]`:第 i 行
- `A[:, j]`:第 j 列
- `A[i1:i2, j1:j2]`:子矩阵

单索引降维 vs 切片保维

- `A[0]`: 一维 (降维)
- `A[0:1]`: 二维 (保维)

视图 vs 副本

- 切片 → 视图 (共享内存)
- 布尔/花式索引 → 副本
- 强制独立副本: `.copy()`

reshape

- `a.reshape(m, n)`: 改形状
- `-1` 自动推断

布尔索引

- `a[a > 0]`: 取满足条件的元素
- `a[a < 0] = 0`: 条件赋值

多条件组合规则

- `&` = AND (不是 `and`)
- `|` = OR (不是 `or`)
- `~` = NOT (不是 `not`)
- 每个条件**必须加括号**

布尔数组的数值技巧

- `mask.sum()`: 数 True 的个数
- `mask.mean()`: True 的比例 → **直接是准确率**

花式索引

- `a[[0, 2, 4]]`: 用整数数组当索引
- 可重复、可乱序

三元运算

- `np.where(cond, x, y)` :满足取 x 否则取 y

排序

- `np.sort(a)` :排序后的数组
- `np.argsort(a)` :升序索引
- `np.argsort(-a)[:k]` :top-k 最大值索引
- `np.argpartition(-a, k)[:k]` :top-k 更快版本

缺失值处理

- `np.isnan(a)` :判断 NaN
- `np.nanmean/nansum/nanstd` :忽略 NaN

第 6 节：广播机制 ★★★★★

核心概念

- 广播 = 把小数组虚拟扩展到大数组的形状
- 不会真复制内存，效率极高
- 让形状不同的数组能直接运算

广播 4 条规则

1. 维度数不同 → 小的前面补 1 直到对齐
2. 每一维大小要么相等，要么其中一个为 1
3. 满足规则 2 → 沿这一维复制对齐
4. 不满足 → 报错

形状判断技巧

- 把两个 shape 右对齐写出来
- 看每一列:相等或为 1 → 能广播
- 否则 → 报错

`np.newaxis` 和 `None` (等价)

- `a[None, :]` :在前面加维度, shape `(1, n)`
- `a[:, None]` :在后面加维度, shape `(n, 1)`
- 用途:让一维数组变行/列向量

维度操作

- `np.expand_dims(a, axis)` :增加维度
- `a.squeeze()` :删除大小为 1 的维度
- `A.transpose(...)` :重排维度 (高维转置)

`keepdims` 参数 (广播好朋友)

- `A.sum(axis=1, keepdims=True)` :聚合后保持维度
- 用途:让聚合结果还能广播回原数组
- 口诀:做完聚合还要广播 → 加 `keepdims=True`

广播经典应用

- 矩阵每行减均值 (标准化)
- 距离矩阵: `X[:, None] - X[None, :]`
- 外积: `a[:, None] * b[None, :]`
- 行归一化: `X / np.linalg.norm(X, axis=1, keepdims=True)`

报错排查 3 步

1. print 两个数组的 shape
 2. 右对齐对比每一维
 3. 找出不匹配的维, 决定 reshape 还是用 newaxis
-

第 7 节：实用工具

数组拼接

- `np.concatenate([A, B], axis=...)`: 通用拼接（不增维）
- `np.vstack([A, B])`: 纵向拼接（等价 `axis=0`）
- `np.hstack([A, B])`: 横向拼接（等价 `axis=1`）
- `np.stack([A, B], axis=...)`: 创建新维度拼接

concatenate vs stack 区别

- `concatenate`: 在已有维度上拼接
- `stack`: 创建新维度

数组分割

- `np.split(arr, n, axis=...)`: 等分（`n` 必须能整除）
- `np.array_split(arr, n)`: 任意分（不能整除也行）
- `np.split(arr, [i1, i2], axis=...)`: 按指定位置分

复制与重复

- `np.tile(a, n)`: 把整个数组重复 `n` 次
- `np.tile(a, (m, n))`: 在多个方向上重复
- `np.repeat(a, n)`: 每个元素重复 `n` 次
- `np.repeat(a, [n1, n2, n3])`: 每个元素重复不同次数

数据 IO

- `np.save('x.npy', arr)`: 保存单个数组（二进制）
- `np.load('x.npy')`: 加载
- `np.savez('x.npz', X=X, y=y)`: 保存多个数组
- `np.savez_compressed`: 压缩保存
- `np.savetxt/loadtxt`: 文本格式（CSV）

实践建议

- 中间结果用 `.npy/.npz` (快、省)
- 给别人看用 `.csv` (通用)

统计常用

- `np.unique(a)` :去重
- `np.unique(a, return_counts=True)` :去重 + 计数
- `np.bincount(a)` :非负整数计数 (更快)
- `np.histogram(data, bins=...)` :直方图

累计计算

- `np.cumsum(a)` :累计求和
- `np.cumprod(a)` :累计乘积

条件查找

- `np.where(condition)` :返回**满足条件的索引**
- `np.any(a)` :至少有一个 True
- `np.all(a)` :全部都是 True

多维最值索引

- `np.unravel_index(A.argmax(), A.shape)` :扁平索引 → 多维索引
- `A.argmax(axis=0/1)` :沿轴的最值索引

截断

- `np.clip(a, min, max)` :把值限制在区间内
- 应用:异常值处理、梯度裁剪

网格点 (画图常用)

- `X, Y = np.meshgrid(x, y)` :生成 2D 网格点
- 应用:等高线、3D 曲面、决策边界

比较两个数组是否相等

- `A == B`: 逐元素比较, 返回**布尔矩阵**
- `(A == B).all()`: 整体相等? 返回布尔值
- `np.array_equal(A, B)`: 整体相等 (整数)
- `np.allclose(A, B)`: 近似相等 (**浮点数推荐**)

常用数学公式一行实现 (必背)

公式	一行实现
Sigmoid $\sigma(x)$	<code>1 / (1 + np.exp(-x))</code>
Softmax	<code>np.exp(x) / np.exp(x).sum()</code>
ReLU	<code>np.maximum(0, x)</code>
L2 范数	<code>np.linalg.norm(x)</code>
余弦相似度	<code>(a @ b) / (np.linalg.norm(a) * np.linalg.norm(b))</code>
Z-score 标准化	<code>(X - X.mean(axis=0)) / X.std(axis=0)</code>
行归一化	<code>X / np.linalg.norm(X, axis=1, keepdims=True)</code>
分类准确率	<code>(y_true == y_pred).mean()</code>
MSE 损失	<code>((y_true - y_pred) ** 2).mean()</code>
最小二乘回归	<code>np.linalg.solve(X.T @ X, X.T @ y)</code>
协方差矩阵	<code>np.cov(X, rowvar=False)</code>
距离矩阵	<code>np.linalg.norm(X[:, None] - X[None, :], axis=-1)</code>
外积	<code>a[:, None] * b[None, :]</code>

9 大核心记忆点

1. NumPy = 张量数据结构 + 高速运算引擎
 2. 形状是一切的基础: `(3, )` `(1, 3)` `(3, 1)` 不同
 3. `*` 是逐元素乘, `@` 是矩阵乘
 4. `axis` 是被消掉的维度
 5. 向量化代替 for 循环
 6. 切片是视图, 需要副本要 `.copy()`
 7. 布尔索引是 NumPy 灵魂
 8. 广播 = 小数组虚拟扩展
 9. 聚合后还要广播 → `keepdims=True`
-

必踩坑提醒（写代码前看一眼）

错	对
<code>np.zeros(3, 4)</code>	<code>np.zeros((3, 4))</code>
<code>A * B</code> (想矩阵乘)	<code>A @ B</code>
<code>(3)</code>	<code>(3,)</code>
<code>axis=0</code> (想要每行的和)	<code>axis=1</code>
<code>(a > 0) and (a < 5)</code>	<code>(a > 0) & (a < 5)</code>
多条件不加括号	每个条件都要加括号
<code>np.linalg.inv(A) @ b</code>	<code>np.linalg.solve(A, b)</code>
<code>eigvecs[i]</code>	<code>eigvecs[:, i]</code>
<code>np.argsort(a, -1:-4)</code>	<code>np.argsort(-a)[:3]</code>
<code>np.dtype(A)</code>	<code>A.dtype</code>
<code>A / A.sum(axis=1)</code>	<code>A / A.sum(axis=1, keepdims=True)</code>
<code>(2, 3) + (2,)</code>	<code>(2, 3) + (2, 1)</code>
<code>np.concatenate(A, B)</code>	<code>np.concatenate([A, B], axis=...)</code>

debug 心法

1. 出 bug 第一件事: `print shape 和 dtype`
2. `SyntaxError` 报第 N 行 → 检查第 N-1 行
3. 维度不对就用 `.reshape()` 或加 `[:, np.newaxis]`
4. 浮点数比较用 `np.allclose`，不用 `==`
5. 广播报错 → 把两个 shape 右对齐看哪一维不匹配
6. 聚合后要再广播 → `keepdims=True`

学完后能做什么

- 独立完成 Kaggle 入门赛的数据预处理
- 看懂任何 NumPy 代码
- 无缝过渡到 Pandas
- 无缝过渡到 PyTorch
- 手撕经典 ML 算法（K-Means、KNN、线性回归、PCA）